

composer/pcre

PCRE wrapping library that offers type-safe `preg_*` replacements.

This library gives you a way to ensure `preg_*` functions do not fail silently, returning unexpected `nulls` that may not be handled.

As of 3.0 this library enforces `PREG_UNMATCHED_AS_NULL` usage for all matching and `replaceCallback` functions, read more below to understand the implications.

It thus makes it easier to work with static analysis tools like PHPStan or Psalm as it simplifies and reduces the possible return values from all the `preg_*` functions which are quite packed with edge cases. As of v2.2.0 / v3.2.0 the library also comes with a PHPStan extension for parsing regular expressions and giving you even better output types.

This library is a thin wrapper around `preg_*` functions with some limitations. If you are looking for a richer API to handle regular expressions have a look at `rawr/t-regx` instead.



Installation

Install the latest version with:

```
$ composer require composer/pcre
```

Requirements

- PHP 7.4.0 is required for 3.x versions
- PHP 7.2.0 is required for 2.x versions
- PHP 5.3.2 is required for 1.x versions

Basic usage

Instead of:

```
if (preg_match('{fo+}', $string, $matches)) { ... }
if (preg_match('{fo+}', $string, $matches, PREG_OFFSET_CAPTURE)) { ... }
if (preg_match_all('{fo+}', $string, $matches)) { ... }
$newString = preg_replace('{fo+}', 'bar', $string);
$newString = preg_replace_callback('{fo+}', function ($match) { return strtoupper($match[0]); }, $string);
$newString = preg_replace_callback_array(['{fo+}' => fn ($match) => strtoupper($match[0])], $string);
$filtered = preg_grep('[a-z]', $elements);
$array = preg_split('[a-z]+', $string);
```

You can now call these on the `Preg` class:

```
use Composer\Pcre\Preg;
```

```
if (Preg::match('{fo+}', $string, $matches)) { ... }
if (Preg::matchWithOffsets('{fo+}', $string, $matches)) { ... }
if (Preg::matchAll('{fo+}', $string, $matches)) { ... }
$newString = Preg::replace('{fo+}', 'bar', $string);
$newString = Preg::replaceCallback('{fo+}', function ($match) { return strtoupper($match[0]); }, $string);
$newString = Preg::replaceCallbackArray(['{fo+}' => fn ($match) => strtoupper($match[0])], $string);
$filtered = Preg::grep('[a-z]', $elements);
$array = Preg::split('[a-z]+', $string);
```

The main difference is if anything fails to match/replace/..., it will throw a `Composer\Pcre\PcreException` instead of returning null (or false in some cases), so you can now use the return values safely relying on the fact that they can only be strings (for replace), ints (for match) or arrays (for grep/split).

Additionally the `Preg` class provides match methods that return `bool` rather than `int`, for stricter type safety when the number of pattern matches is not useful:

```
use Composer\Pcre\Preg;
```

```
if (Preg::isMatch('{fo+}', $string, $matches)) // bool
if (Preg::isMatchAll('{fo+}', $string, $matches)) // bool
```

Finally the `Preg` class provides a few `*StrictGroups` method variants that ensure match groups are always present and thus non-nullable, making it easier to write type-safe code:

```
use Composer\Pcre\Preg;
```

```
// $matches is guaranteed to be an array of strings, if a subpattern does not match and produce an empty string
if (Preg::matchStrictGroups('{fo+}', $string, $matches))
if (Preg::matchAllStrictGroups('{fo+}', $string, $matches))
```

Note: This is generally safe to use as long as you do not have optional subpatterns (i.e. `(something)?` or `(something)*` or branches with a `|` that result in some groups not being matched at all). A subpattern that can match an empty string like `(.*)` is **not** optional, it will be present as an empty string in the matches. A non-matching subpattern, even if optional like `(?:foo)?` will anyway not be present in matches so it is also not a problem to use these with `*StrictGroups` methods.

If you would prefer a slightly more verbose usage, replacing by-ref arguments by result objects, you can use the `Regex` class:

```
use Composer\Pcre\Regex;
```

```
// this is useful when you are just interested in knowing if something matched
// as it returns a bool instead of int(1/0) for match
```

```

$bool = Regex::isMatch('{fo+}', $string);

$result = Regex::match('{fo+}', $string);
if ($result->matched) { something($result->matches); }

$result = Regex::matchWithOffsets('{fo+}', $string);
if ($result->matched) { something($result->matches); }

$result = Regex::matchAll('{fo+}', $string);
if ($result->matched && $result->count > 3) { something($result->matches); }

$newString = Regex::replace('{fo+}', 'bar', $string)->result;
$newString = Regex::replaceCallback('{fo+}', function ($match) { return strtoupper($match[0]); }, $string);
$newString = Regex::replaceCallbackArray(['{fo+}' => fn ($match) => strtoupper($match[0])], $string);

```

Note that `preg_grep` and `preg_split` are only callable via the `Preg` class as they do not have complex return types warranting a specific result object.

See the `MatchResult`, `MatchWithOffsetsResult`, `MatchAllResult`, `MatchAllWithOffsetsResult`, and `ReplaceResult` class sources for more details.

Restrictions / Limitations

Due to type safety requirements a few restrictions are in place.

- matching using `PREG_OFFSET_CAPTURE` is made available via `matchWithOffsets` and `matchAllWithOffsets`. You cannot pass the flag to `match/matchAll`.
- `Preg::split` will also reject `PREG_SPLIT_OFFSET_CAPTURE` and you should use `splitWithOffsets` instead.
- `matchAll` rejects `PREG_SET_ORDER` as it also changes the shape of the returned matches. There is no alternative provided as you can fairly easily code around it.
- `preg_filter` is not supported as it has a rather crazy API, most likely you should rather use `Preg::grep` in combination with some loop and `Preg::replace`.
- `replace`, `replaceCallback` and `replaceCallbackArray` do not support an array `$subject`, only simple strings.
- As of 2.0, the library always uses `PREG_UNMATCHED_AS_NULL` for matching, which offers much saner/more predictable results. As of 3.0 the flag is also set for `replaceCallback` and `replaceCallbackArray`.

PREG_UNMATCHED_AS_NULL As of 2.0, this library always uses `PREG_UNMATCHED_AS_NULL` for all `match*` and `isMatch*` functions. As of 3.0 it is also done for `replaceCallback` and `replaceCallbackArray`.

This means your matches will always contain all matching groups, either as null if unmatched or as string if it matched.

The advantages in clarity and predictability are clearer if you compare the two outputs of running this with and without `PREG_UNMATCHED_AS_NULL` in `$flags`:

```
preg_match('/(a)(b)*(c)(d)*/', 'ac', $matches, $flags);
```

no flag	PREG_UNMATCHED_AS_NULL
array (size=4)	array (size=5)
0 => string 'ac' (length=2)	0 => string 'ac' (length=2)
1 => string 'a' (length=1)	1 => string 'a' (length=1)
2 => string '' (length=0)	2 => null
3 => string 'c' (length=1)	3 => string 'c' (length=1)
	4 => null
group 2 (any unmatched group preceding one that matched) is set to ''. You cannot tell if it matched an empty string or did not match at all	group 2 is <code>null</code> when unmatched and a string if it matched, easy to check for
group 4 (any optional group without a matching one following) is missing altogether. So you have to check with <code>isset()</code> , but really you want <code>isset(\$m[4]) && \$m[4] !== ''</code> for safety unless you are very careful to check that a non-optional group follows it	group 4 is always set, and null in this case as there was no match, easy to check for with <code>\$m[4] !== null</code>

PHPStan Extension

To use the PHPStan extension if you do not use `phpstan/extension-installer` you can include `vendor/composer/pcre/extension.neon` in your PHPStan config.

The extension provides much better type information for `$matches` as well as regex validation where possible.

License

`composer/pcre` is licensed under the MIT License, see the `LICENSE` file for details.